



Already have an account? [Sign in](#) →

行程表 (暫定)

W	Date	Lecture	Homework	TA Class
1	09/10, 09/11	Course Information / No class - Registration	Homework 00 (Wed.)	V
2	09/17, 09/18	Git Introduction		
3	09/24, 09/25	OOP Concept	Homework 01 (Wed.)	V
4	10/01, 10/02	Introducing What C++ is		
5	10/08, 10/09	Class / Essential STL Introduction	Homework 02 (Wed.)	
6	10/15, 10/16	Lec03: Encapsulation		V
7	10/22, 10/23	Lec04: Inheritance	Homework 03 (Wed.)	
8	10/29, 10/30	Lec04: Inheritance		V
9	11/05, 11/06	Physical Hand-Written Midterm	Homework 04 (Mon.)	
10	11/12, 11/13	Physical Computer-based Midterm		
11	11/19, 11/20	Lec05: Polymorphism	Homework 05 (Wed.)	V
12	11/26, 12/27	Lec05: Polymorphism		
13	12/03, 12/04	Lec05: Polymorphism - LAB	Homework 06 (Wed.)	V
14	12/10, 12/11	Lec06: Composition & Interface		
15	12/17, 12/18	Lec06: Composition & Interface	Homework 07 (Wed.)	
16	12/24, 12/25	Lec06: Composition & Interface - Lab/ No class		V
17	12/31, 01/01	Physical Hand-Written Final / No class		
18	01/07, 01/08	Lec07: Experience sharing & OOPL - Preparation / Physical Computer-based Final		

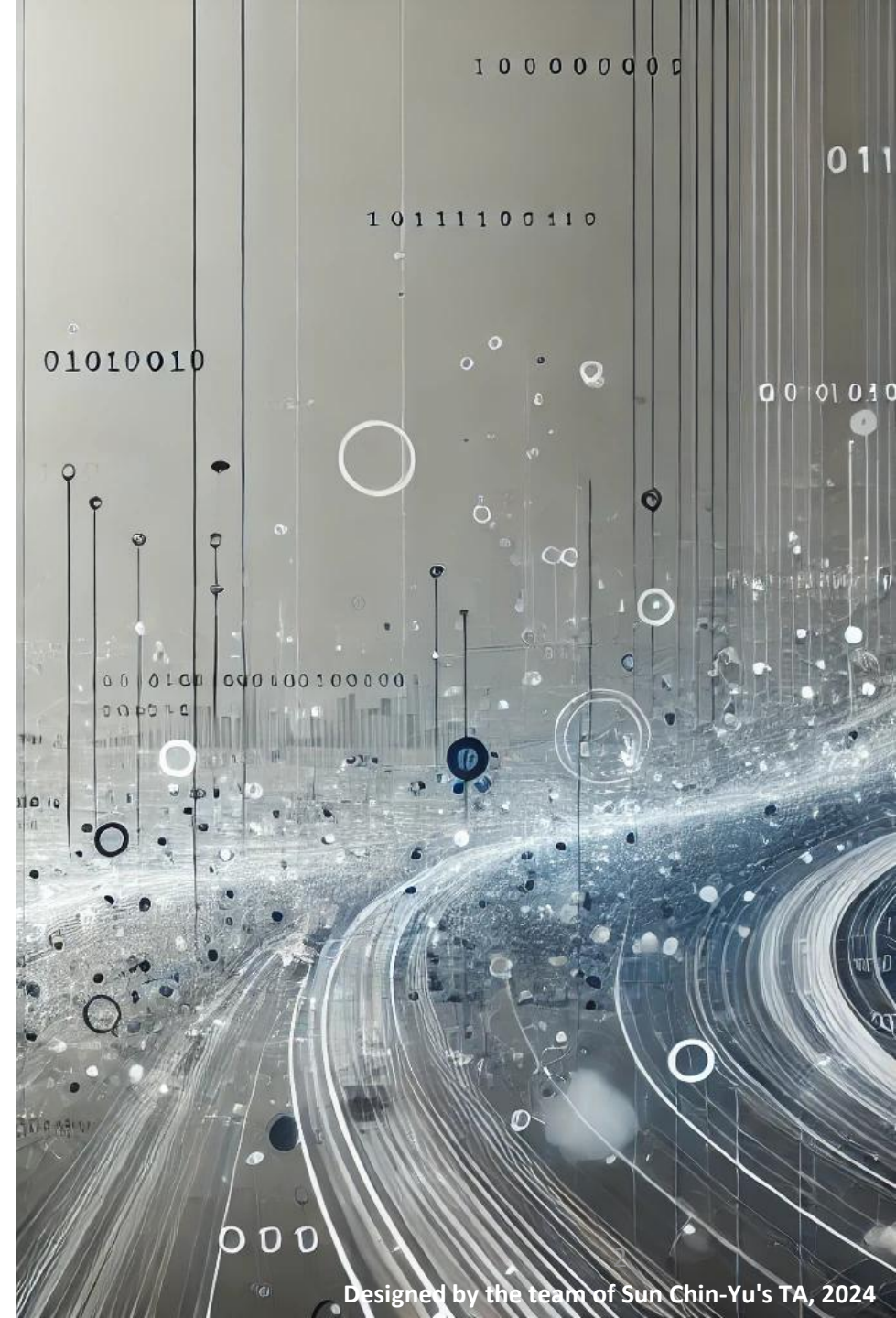
物件導向程式設計

第參章：C++ 簡介

授課講師：孫勤昱博士

國立臺北科技大學 資訊工程系

cysun@ntut.edu.tw



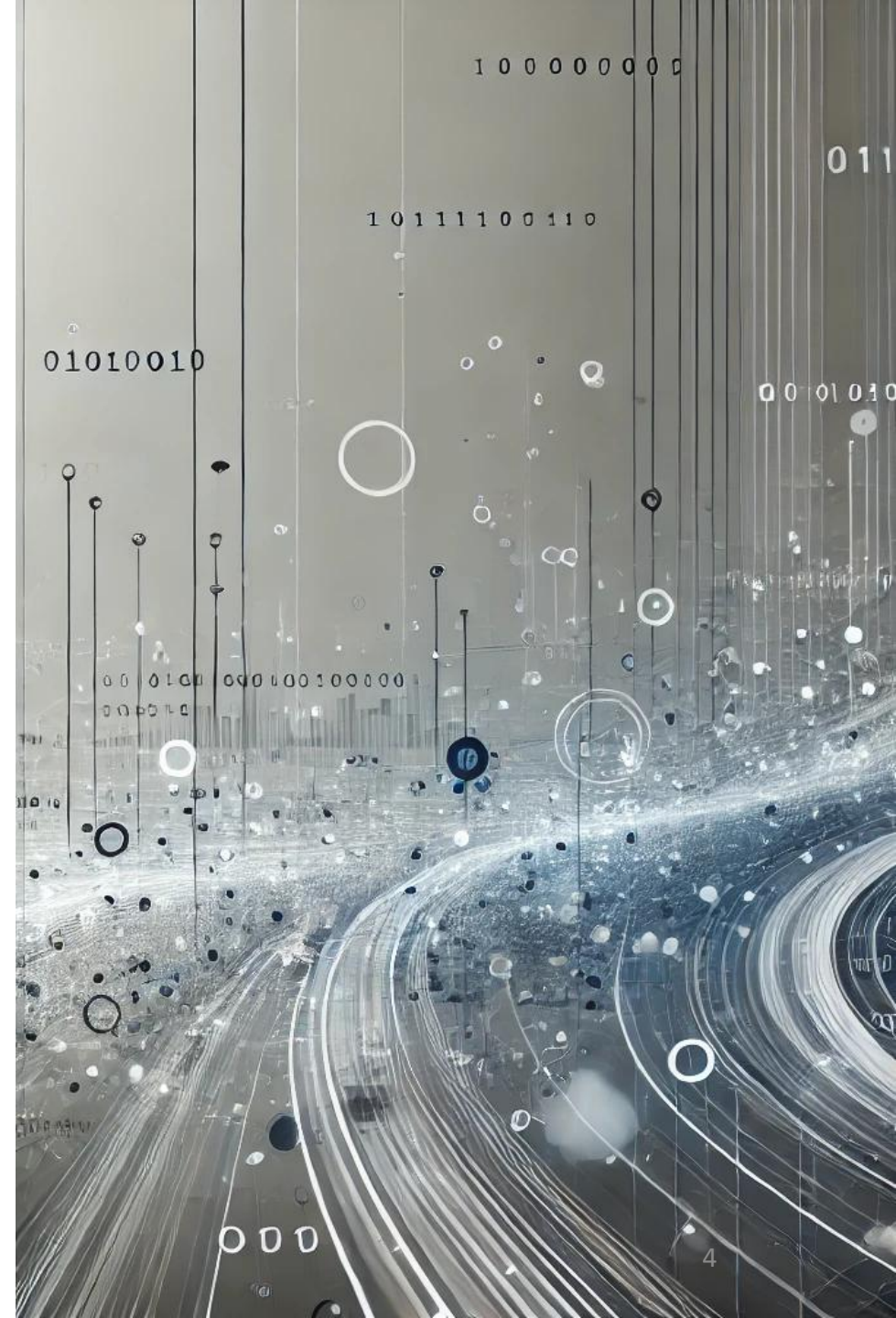
Outline

- C++ 的版本介紹
- 基本的C++語法
 - Print & Scan
 - String
 - If condition ...
- 介紹主流的 Compiler
- Compiler 編譯的步驟
- 標頭檔與預編譯



主題一：

什麼是 C++ 的版本？



什麼是 C++ 的版本 (1/6)

- 指的是「C++ 標準」的版本
 - 由國際標準化組織 (International Organization for Standardization, ISO) 制定的官方規範
 - ISO/IEC 14882:1998
 - 它決定了程式語言的語法、特性和功能的規範
- 舉個例子, Python 3.x 的版本
 - 版本之間的向下兼容性: 較新版本的語言仍然可以執行較早版本所寫的程式碼
 - 特例 Python 2 -> 3
 - 需要進行一些程式碼修改



什麼是 C++ 的版本 (2/6)

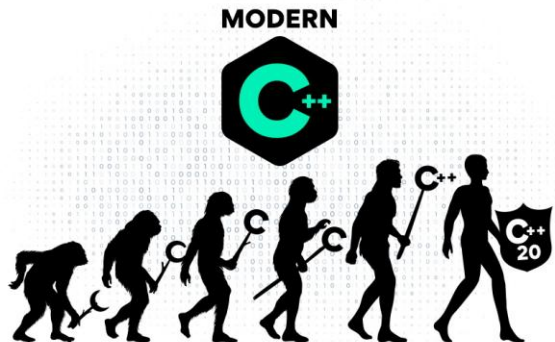
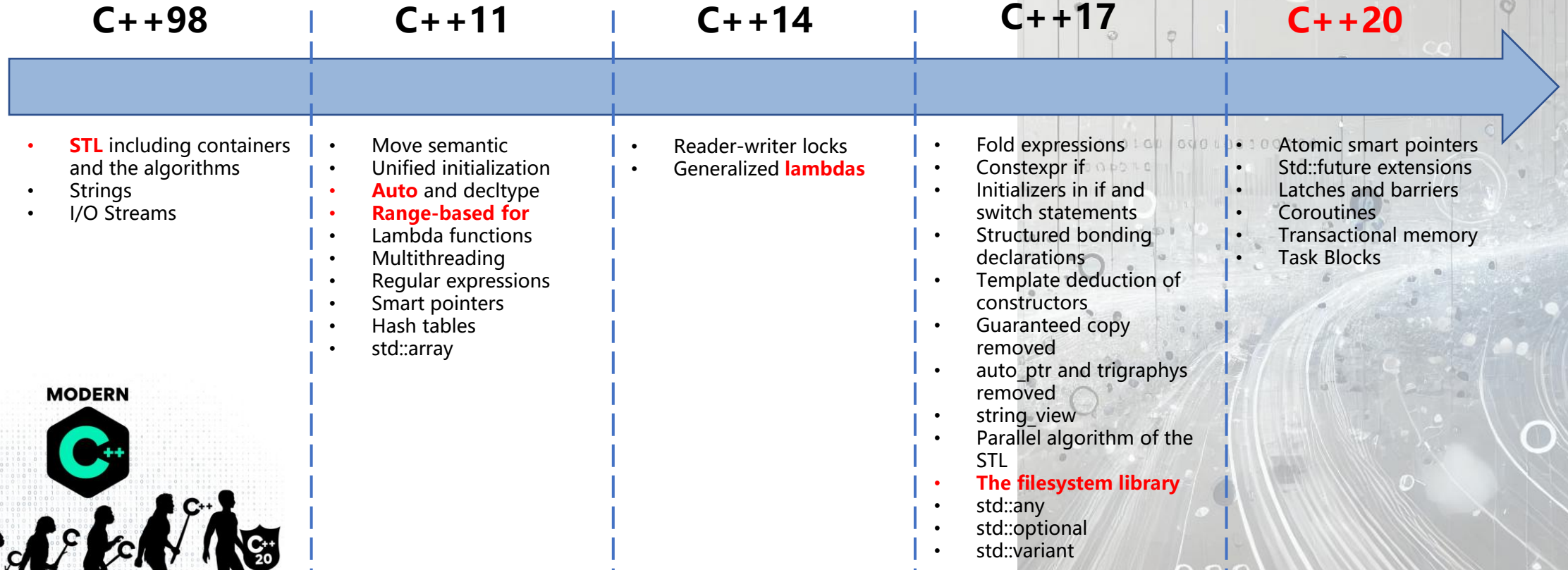
- C 是如何演化到 Modern C++
 - 解決 C 的記憶體操作的複雜性與 “C with Classes”^[1]
 - 有基本的 Constructors and Destructors
 - Overloading
 - Public/Private access control
 - Derived classes
 - ...等
 - 後面幾個重大的改版
 - C++98, C++11, ...



Bjarne Stroustrup

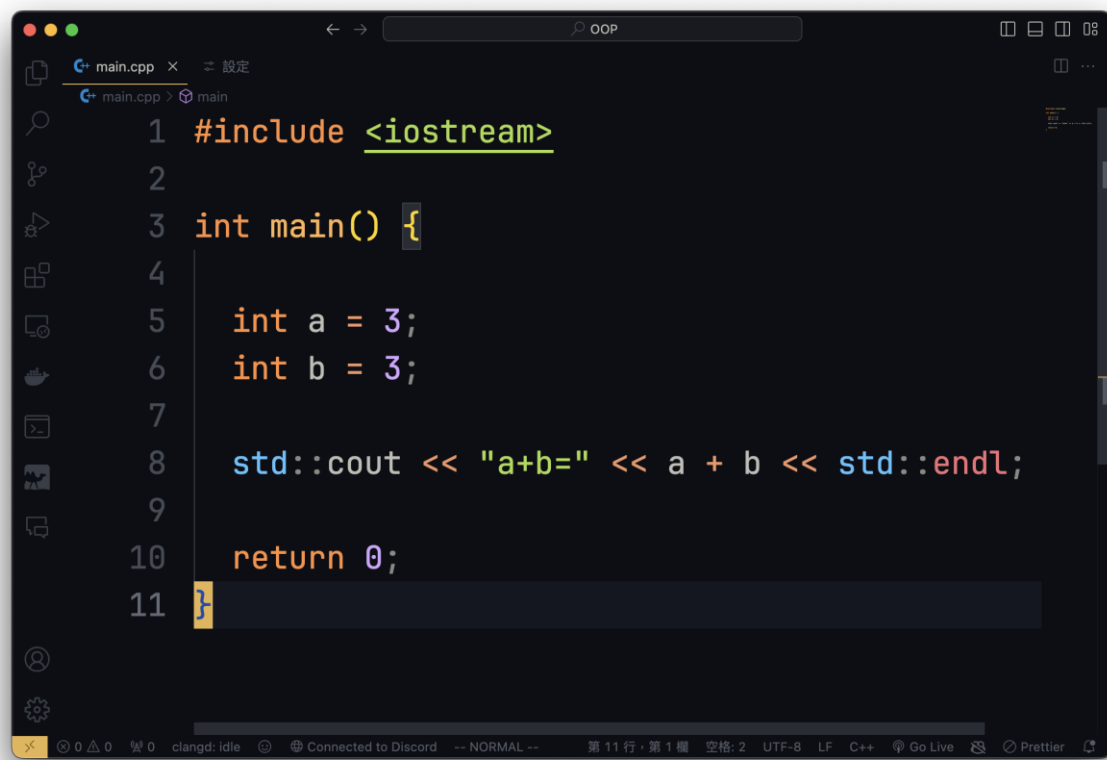
什麼是 C++ 的版本 (3/6)

- C++ 版本發展簡史

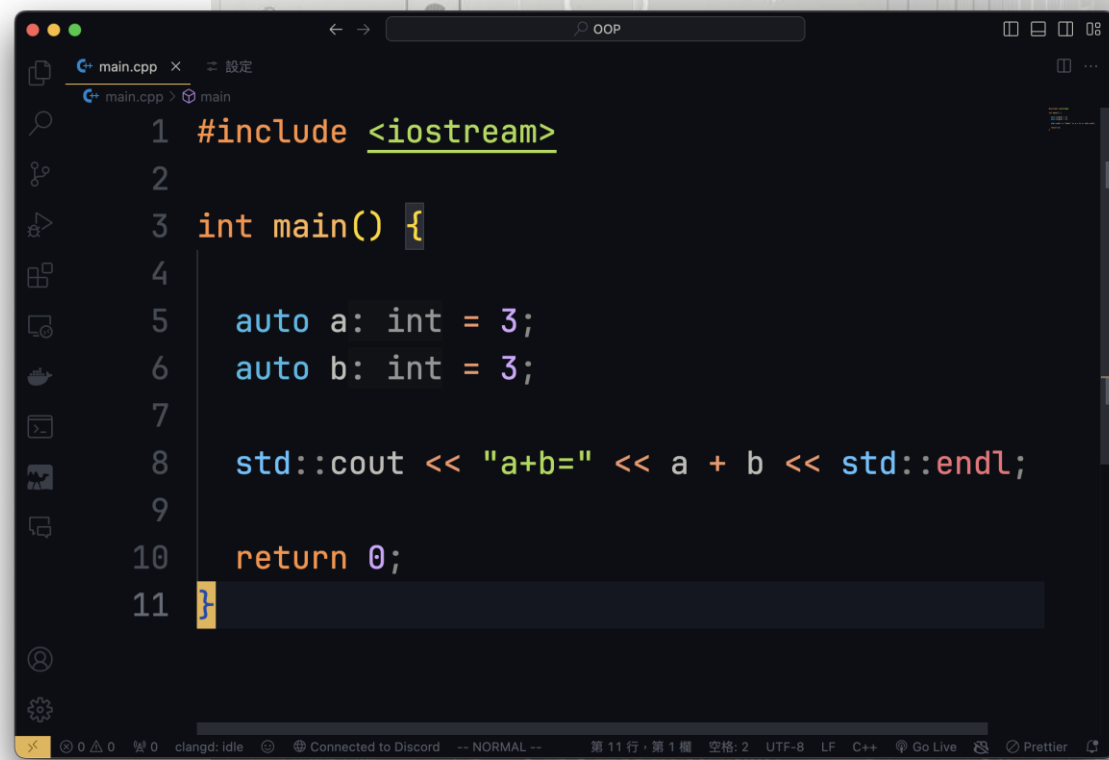


什麼是 C++ 的版本 (4/6)

- 酷酷的新語法 - auto 篇
 - auto 是可以取代型別的撰寫，讓編譯器做型別推論



```
1 #include <iostream>
2
3 int main() {
4
5     int a = 3;
6     int b = 3;
7
8     std::cout << "a+b=" << a + b << std::endl;
9
10    return 0;
11 }
```

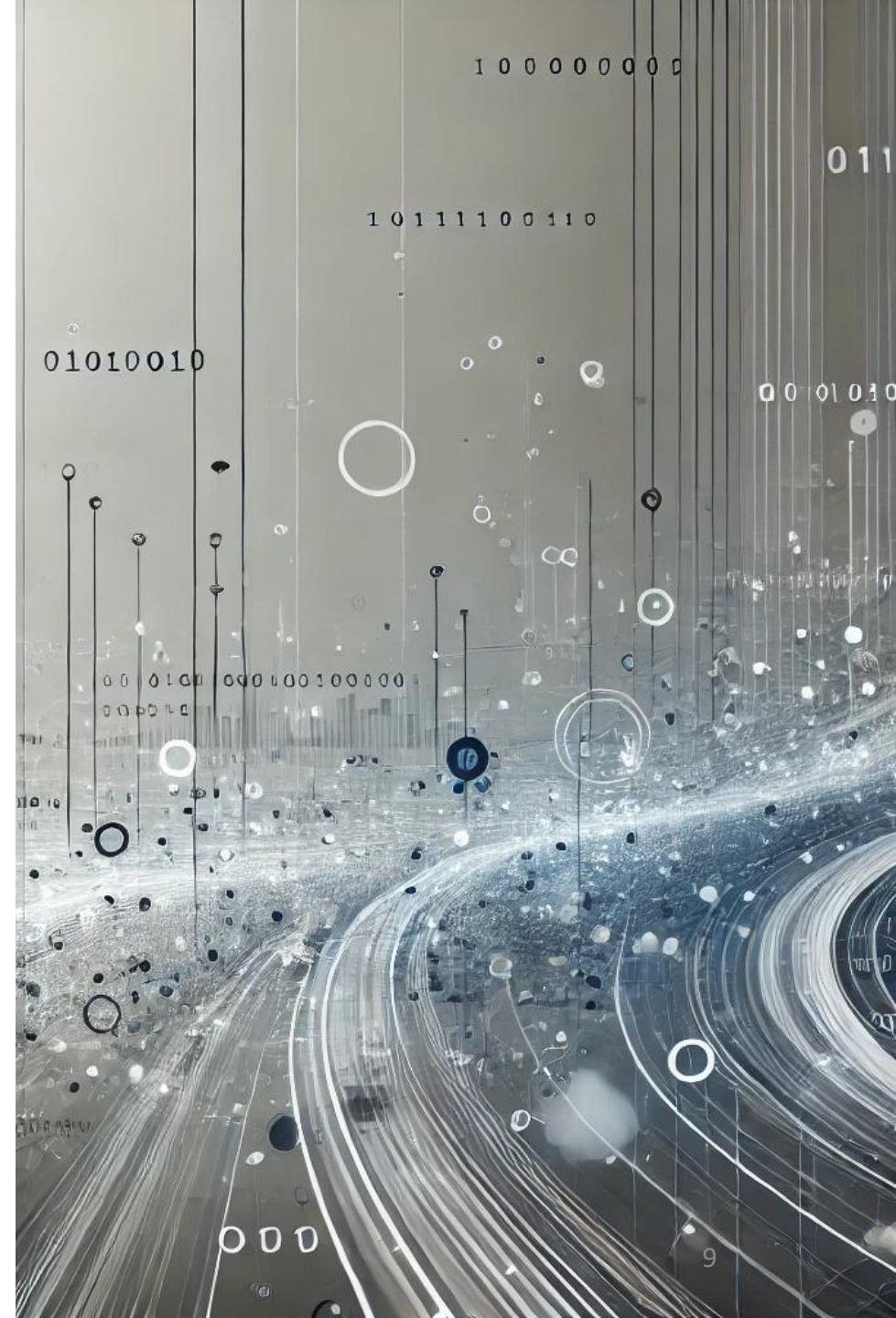


```
1 #include <iostream>
2
3 int main() {
4
5     auto a: int = 3;
6     auto b: int = 3;
7
8     std::cout << "a+b=" << a + b << std::endl;
9
10    return 0;
11 }
```


主題二：

C++ 語法 - 基本

從輸入輸出到指標參考 一天學廢



C++ 語法 - 基本 輸出文字篇

- 在 C 使用 printf 輸出文字至終端機

```
● ● ●  
  
#include <stdio.h>  
  
int main() {  
    printf("Hello, world");  
    return 0;  
}
```

- 在 C++ 使用 std::cout 輸出文字至終端機

```
● ● ●  
  
#include <iostream>  
  
int main() {  
    std::cout << "Hello, world";  
    return 0;  
}
```



C++ 語法 - 基本 輸出文字篇

- 使用 C 語言來輸出時，需要使用正確的类型別

```
● ● ●  
  
#include <stdio.h>  
  
int main() {  
    int a = 48763;  
    printf("%d",a);  
    return 0;  
}
```

- 在 C++ 語言中，不需要管輸出的类型別

```
● ● ●  
  
#include <iostream>  
  
int main() {  
    int a = 48763;  
    std::cout<<a;  
    return 0;  
}
```



C++ 語法 - 基本 問題思考篇

- 輸出的型別不匹配會發生什麼事嗎？



```
#include <stdio.h>

int main() {

    printf("%d\n", 123);
    printf("%s\n", "test");

}
```



```
#include <stdio.h>

int main() {

    printf("%s\n", 123);

}
```



C++ 語法 - 基本 問題思考篇

- 輸出的型別不匹配會發生什麼事嗎？
 - 編譯器對printf()中的 %s 解析，會期待接收到抓一個 char*，也就是記憶體位址
 - 而這個 整數123 對編譯器而言變成記憶體位置的時候，它會去這個記憶體位置去抓東西
 - 變成非法存取到配置的記憶體空間



The screenshot shows a C++ IDE with a single file named 07.c. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     printf("%s\n", 123);
5     return 0;
6 }
```

The IDE's output window at the bottom shows the execution of the program, which results in a segmentation fault:

```
test ./main
[1] 77806 segmentation fault ./main
test
```

The status bar at the bottom indicates the compiler is clangd, the file is 07.c, and the encoding is UTF-8.

C++ 語法 - 基本 文字換行篇

- 在終端機顯示「換行」

```
#include <iostream>

int main() {

    std::cout << "I like OOP" << "\n" ;

    return 0;
}
```

```
#include <iostream>

int main() {

    std::cout << "I like OOP" << std::endl;

    return 0;
}
```

- `\n` 是一個字元 (newline character) , 它直接插入到輸出的字串中, 代表一個換行字元。它相當於告訴編譯器, 在這個位置換行。
- `std::endl` 是一個操作, 它不僅插入一個換行字元, 還會強制調用 `flush()` , 這樣緩衝區的內容會立即寫到終端或文件中 (印出) 。

C++ 語法 - 基本 問題思考篇

- 下面這段程式碼，適合用 `\n` 還是 `std::endl`？請說明你的答案

```
main.cpp
1  #include <iostream>
2
3  int main(){
4      for (int i = 0; i < 1000; i++) {
5          std::cout << i << _____;
6      }
7  }
```

- `\n` 的情境：一般輸出換行、大量輸出 (例如迴圈、輸出檔案)
- `endl` 的情境：提示使用者輸入、互動式程式、debug



C++ 語法 - 基本 文字輸入篇

- C 語言的輸入語法 v.s. C++ 語言的輸入語法

特色	C語言(scanf)	C++語言 (cin)
語法	使用函數呼叫，格式化字串	使用運算子多載 >>
安全性	易於發生緩衝區溢出或輸入錯誤	較C語言安全
執行效能	快	慢
資料型別處理	必須手動指定格式化符號	自動處理資料型別
錯誤處理機制	無法直接處理格式錯誤	能檢查輸入錯誤

```
#include <stdio.h>

int main() {

    int a;
    scanf("%d",&a);

    return 0;
}
```

C 語言的輸入語法

```
#include <iostream>

int main() {

    int a;
    std::cin >> a;

    return 0;
}
```

C++ 語言的輸入語法

C++ 語法 - 基本 註解篇

- 註解是給程式設計師看的一段文字訊息
 - 編譯器會忽略此訊息
- 單行註解 (Line comments)
 - 兩條斜線 // 後接該行的任何內容
- 多行註解 (Block comments)
 - 用 /* 和 */ 括住, 且通常可以跨越多行

```
// std::cout << "I like OOP";

/*
int main() {
    std::cout << "Hello, world";
    return 0;
}*/
```

C++ 語法 - 基本 字串篇

- 初始化字串 與 長度計算



```
// C:  
char *str = (char *)malloc(sizeof(char) * 5);  
strncpy(str, "test", 5);  
  
strlen(str); // 計算字串長度  
free(str)
```



```
// C++:  
std::string str = "test";  
  
str.length(); // 計算長度
```

- 在 C++ 中 std::string 所需的記憶體，都會自動的分配與釋放

C++ 語法 - 基本 問題思考篇

- 在 C 當中創建 String 時，有可能會遇到什麼問題嗎？



```
// Problem:  
char *str = (char*)malloc(sizeof(char) * 4);  
strncpy(str, "test", 4);  
  
printf("%s\n", str);  
  
free(str);
```

C++ 語法 - 基本 問題思考篇

- 在 C 當中創建 String 時，有可能會遇到什麼問題嗎



```
// Problem:  
char *str = (char*)malloc(sizeof(char) * 4);  
strncpy(str, "test", 4);  
  
printf("%s\n", str);  
  
free(str);
```

- 將 test 複製進去 *str 時，缺少了終止字元 \0，這會導致 printf 函數讀到**非預期的長度**
- C 語言存在這很多這樣類似的問題
 - 安全程式設計 (研究所)

C++ 語法 - 基本 字串篇

- 字串串接



```
char* a = "hello";  
char* b = "world";  
char ans[64] = {0};
```

```
strcat(ans,a);  
strcat(ans," ");  
strcat(ans,b);
```



```
std::string a = "hello";  
std::string b = "world";
```

```
a = a + " " + b;
```



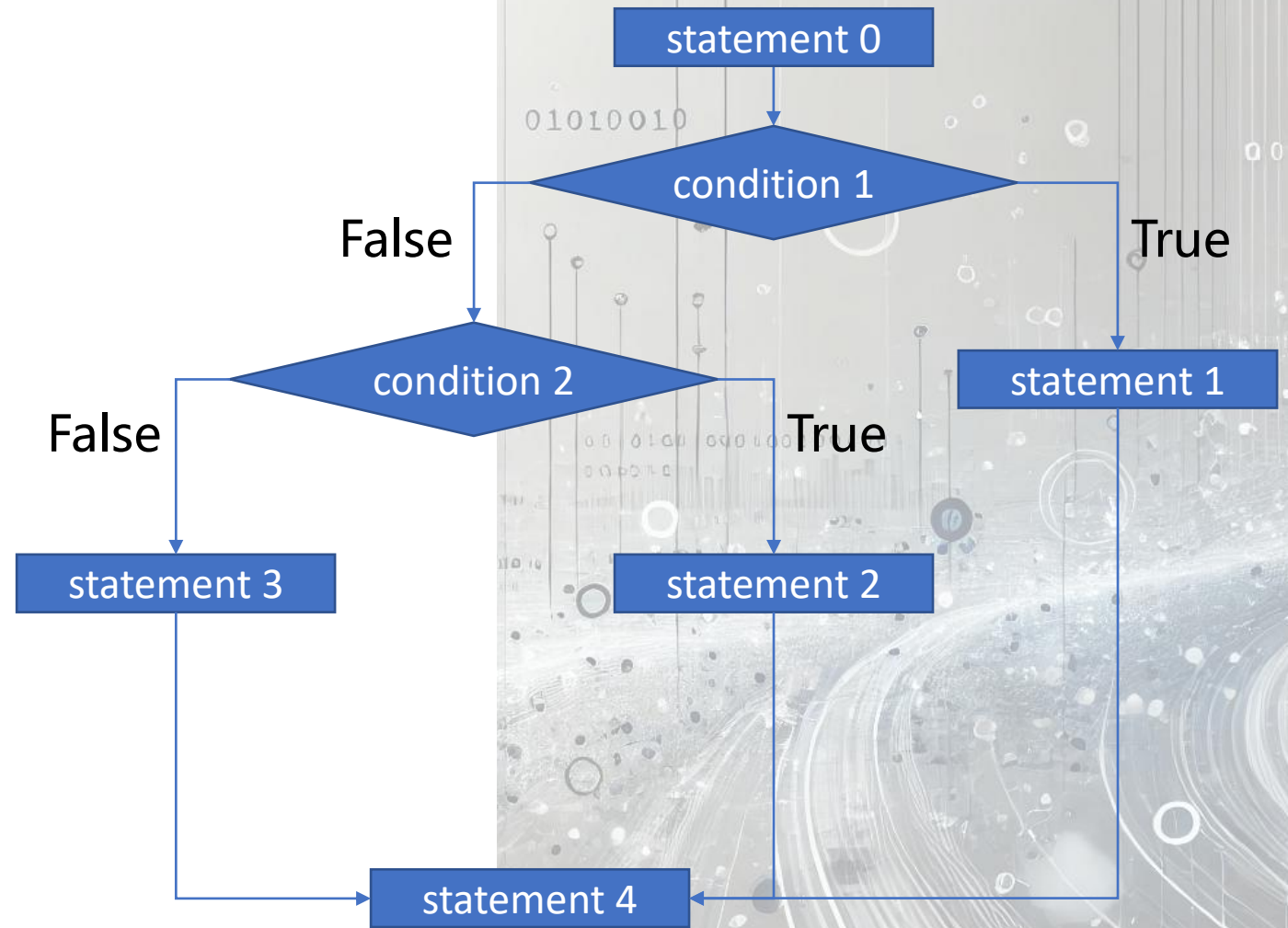
C++ 語法 - 基本 條件判斷篇

- 條件判斷

```
main.cpp
1 statement 0;
2 if (condition 1){
3     statement 1;
4 }else if (condition 2){
5     statement 2;
6 }else{
7     statement 3;
8 }
9 statement 4;
```

條件句

陳述句



C++ 語法 - 基本 條件判斷篇

- 三元運算子 (Ternary operator)
 - $a ? b : c$
 - 當 a 為真則執行 b , 當 a 為否則執行 c

```
1 // js, cpp, java, ...  
2 return condition ? 0 : 1
```

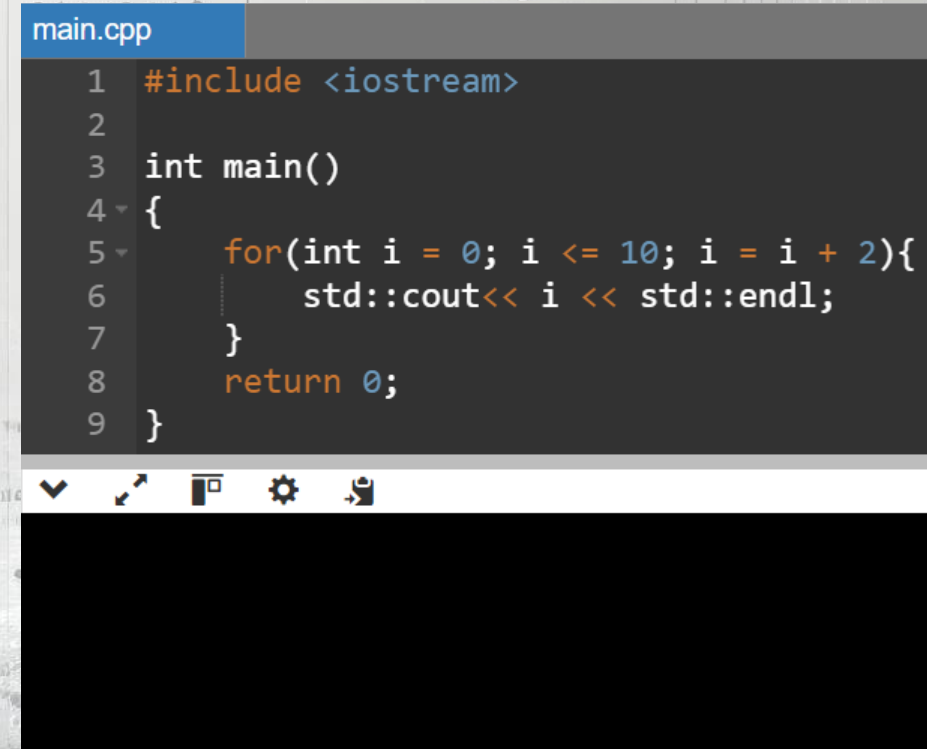
=

```
1 if (condition) {  
2     return 0;  
3 } else {  
4     return 1;  
5 }
```

- Exercise:
 - Score如果及格(60↑、含60)的話 顯示通過
 - Score如果不及格(60↓)的畫 顯示不通過
 -

C++ 語法 - 基本 迴圈篇

- For 迴圈
 - 利用索引變數值的累加或遞減控制迴圈執行次數，等該變數值達到設定標準，就會跳出迴圈
- 語法
 - for (初始值設定; 終止條件判斷; 每次迴圈後的運算) {
 迴圈內的指令動作
}
 - 初始值設定：設定條件式中用到變數之初始值
 - 終止條件判斷：判斷是否執行迴圈中的程式
 - 每次迴圈後的運算：用於調整條件算式中的變數值



```
main.cpp
1  #include <iostream>
2
3  int main()
4  {
5      for(int i = 0; i <= 10; i = i + 2){
6          std::cout<< i << std::endl;
7      }
8      return 0;
9  }
```


C++ 語法 - 基本 陣列篇

- 陣列 (Vector) 是一種用來儲存資料的資料結構
 - 陣列中的每個元素 (Element) 都是相同的資料型別
 - 利用索引值 (Index) 就可以找出對應的元素
- 在 C++ 裡面創建一個陣列, 用 `std::vector`
 - 成員函式: `size()`、`resize()`、`empty()`、`front()`、`back()`、...

```
// std::vector<型別> 變數名 = 資料  
  
std::vector<int> list = {1, 2, 3};  
  
arr.size() // 取得陣列的長度
```

C++ 語法 - 基本 迴圈篇

- 使用陣列來儲存五個string元素
 - Apple, Banana, Book, OOP, 和 Teacher
- 並用for迴圈的方式逐行印出這些元素

main.cpp

```
1 #include <iostream>
2
3 int main()
4 {
5     std::string str[5] = {"Apple", "Banana", "Book", "OOP", "Teacher"};
6
7     for (int i = 0; i < std::size_t(str); i++){
8         std::cout << str[i] << std::endl;
9     }
10 }
```

main.cpp

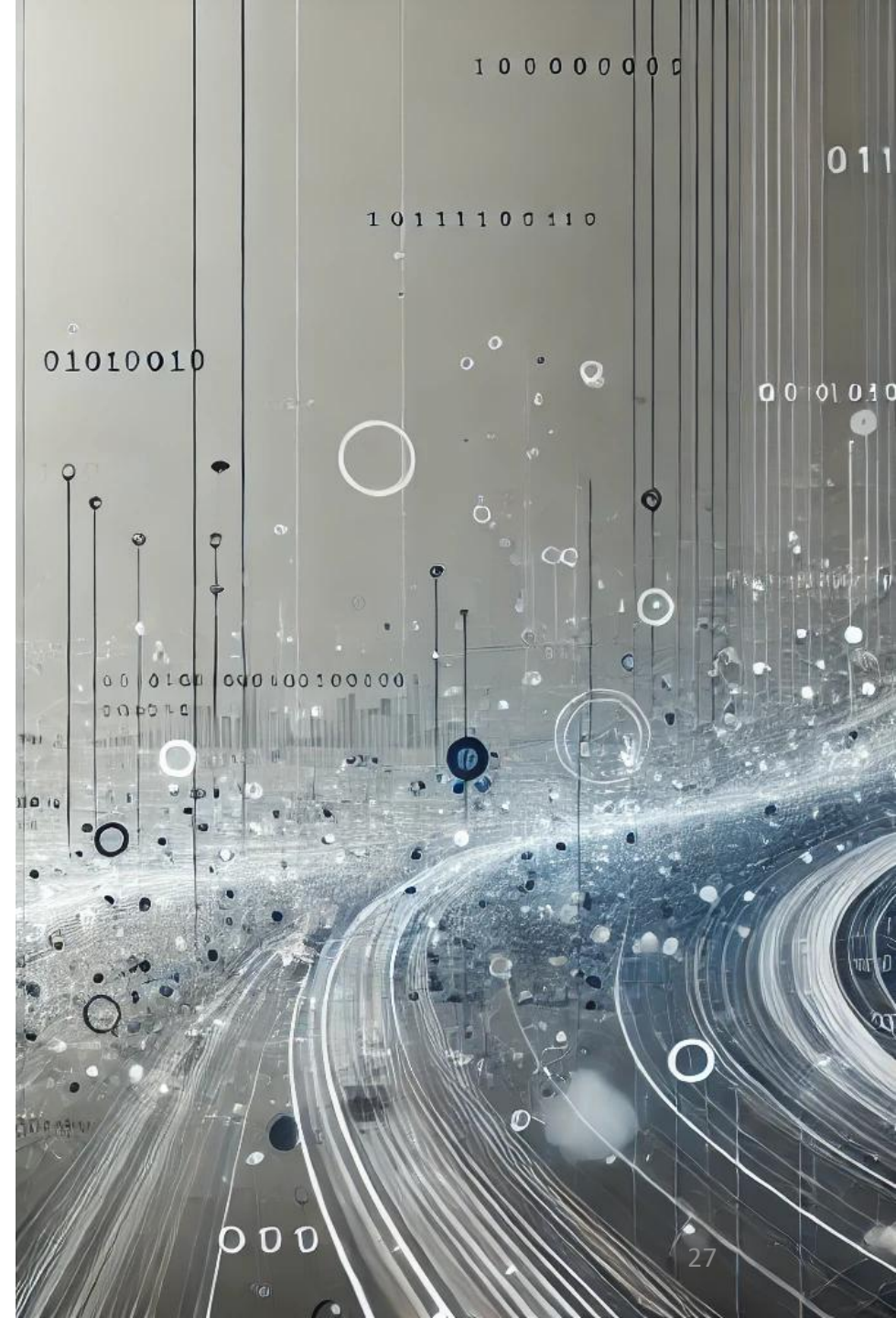
```
1 #include <iostream>
2
3 int main()
4 {
5     std::string str[5] = {"Apple", "Banana", "Book", "OOP", "Teacher"};
6
7     for (const auto& s : str) {
8         std::cout << s << std::endl;
9     }
10 }
```

Range-based for loop

主題三：

C++ 語法 - 基本二

從輸入輸出到指標參考 一天學廢



C++ 語法 - 基本 Switch-case篇

```
switch(條件算式) {  
    case 條件算式值1:  
        動作1;  
        break;  
    case 條件算式值2:  
        動作2;  
        break;  
    default:  
        預設動作;  
        break;  
}
```

- 從數個Case中，選出一個來執行
- 語法：
 - 條件算式：結果為數值或字元的，也可以是變數名稱
 - 條件算式值：根據條件算式的輸入值來判斷該執行哪個 case
 - case：在 switch 內的敘述，每個 case 都有不同的條件算式值與動作
 - break：結束 case 動作
 - default：預設選項，當 switch 找不到符合的 case，便會執行 default 內的程式碼

C++ 語法 - 基本 Switch-case篇

- 程式範例

```
1  #include <stdio.h>
2
3  void func(int state) {
4      switch (state) {
5          case 0:
6              printf("POWER_ON\n");
7              break;
8          case 1:
9              printf("POWER_OFF\n");
10             break;
11          case 2:
12              printf("SLEEP\n");
13              break;
14          case 3:
15              printf("LOCK\n");
16              break;
17          case 4:
18              printf("RESTART\n");
19              break;
20
21      }
22
23  }
24 }
```

請思考一下，這裡可能存在什麼潛在問題？ 29

C++ 語法 - 基本 Switch-case篇

- 問題一、輸入值的範圍

```
1  #include <stdio.h>
2
3  void func(int state) {
4      switch (state) {
5          case 0:
6              printf("POWER_ON\n");
7              break;
8          case 1:
9              printf("POWER_OFF\n");
10             break;
11          case 2:
12              printf("SLEEP\n");
13              break;
14          case 3:
15              printf("LOCK\n");
16              break;
17          case 4:
18              printf("RESTART\n");
19              break;
20          default:
21              printf("Unknown state\n");
22              break;
23      }
24  }
```

- state 是一個 int 型別，理論上可以接受任何整數
- 如果 state 超出 0、1、2 的範圍，則會進入 default 區塊
 - 這本身並不是錯誤，但應該確保 state 的值符合預期。如果此函數會在多種情況下被呼叫，可能需要檢查 state 是否落在可接受的範圍內
- 如果沒有設定 default 區塊
 - 靜默失效 (Silent failure)

C++ 語法 - 基本 Switch-case篇

- 問題二、可讀性非常差

```
1  #include <stdio.h>
2
3  void func(int state) {
4      switch (state) {
5          case 0:
6              printf("POWER_ON\n");
7              break;
8          case 1:
9              printf("POWER_OFF\n");
10             break;
11          case 2:
12              printf("SLEEP\n");
13              break;
14          case 3:
15              printf("LOCK\n");
16              break;
17          case 4:
18              printf("RESTART\n");
19              break;
20          default:
21              printf("Unknown state\n");
22              break;
23      }
24  }
```

- 不知道 0, 1, ..., 4 分別代表什麼意思

```
26 int main() {
27     func(0);
28     func(1);
29     func(2);
30     func(3);
31     func(4);
32
33     return 0;
34 }
```

C++ 語法 - 基本 Enum篇

- 列舉，比較適合的一種寫法

```
1  #include <stdio.h>
2
3  enum STATE {
4      POWER_ON = 0,
5      POWER_OFF,
6      SLEEP,
7      LOCK,
8      RESTART
9  };
10
11 void func(enum STATE state) {
12     switch (state) {
13         case POWER_ON:
14             printf("Execution01: POWER_ON\n");
15             break;
16         case POWER_OFF:
17             printf("Execution02: POWER_OFF\n");
18             break;
19         case SLEEP:
20             printf("Execution03: SLEEP\n");
21             break;
22         case LOCK:
23             printf("Execution04: LOCK\n");
24             break;
25         case RESTART:
26             printf("Execution05: RESTART\n");
27             break;
28         default:
29             printf("Unknown state\n");
30             break;
31     }
32 }
```

```
34 int main() {
35     func(POWER_ON);
36     func(POWER_OFF);
37     func(SLEEP);
38     func(LOCK);
39     func(RESTART);
40
41     return 0;
42 }
```

- Enum 的語法

```
// Enum 語法
Enum STATE {
    POWER_ON = 0,
    POWER_OFF,
    SLEEP,
    LOCK,
    RESTART,
}
```

- 等價於.....

```
// 等價於
const int POWER_ON = 0;
const int POWER_OFF = 1;
const int SLEEP = 2;
const int LOCK = 3;
const int RESTART = 4;
```


C++ 語法 - 基本 問題思考篇

- 問題出現了!



C++ 語法 - 基本 Enum Class篇

- 使用 Enum Class.....

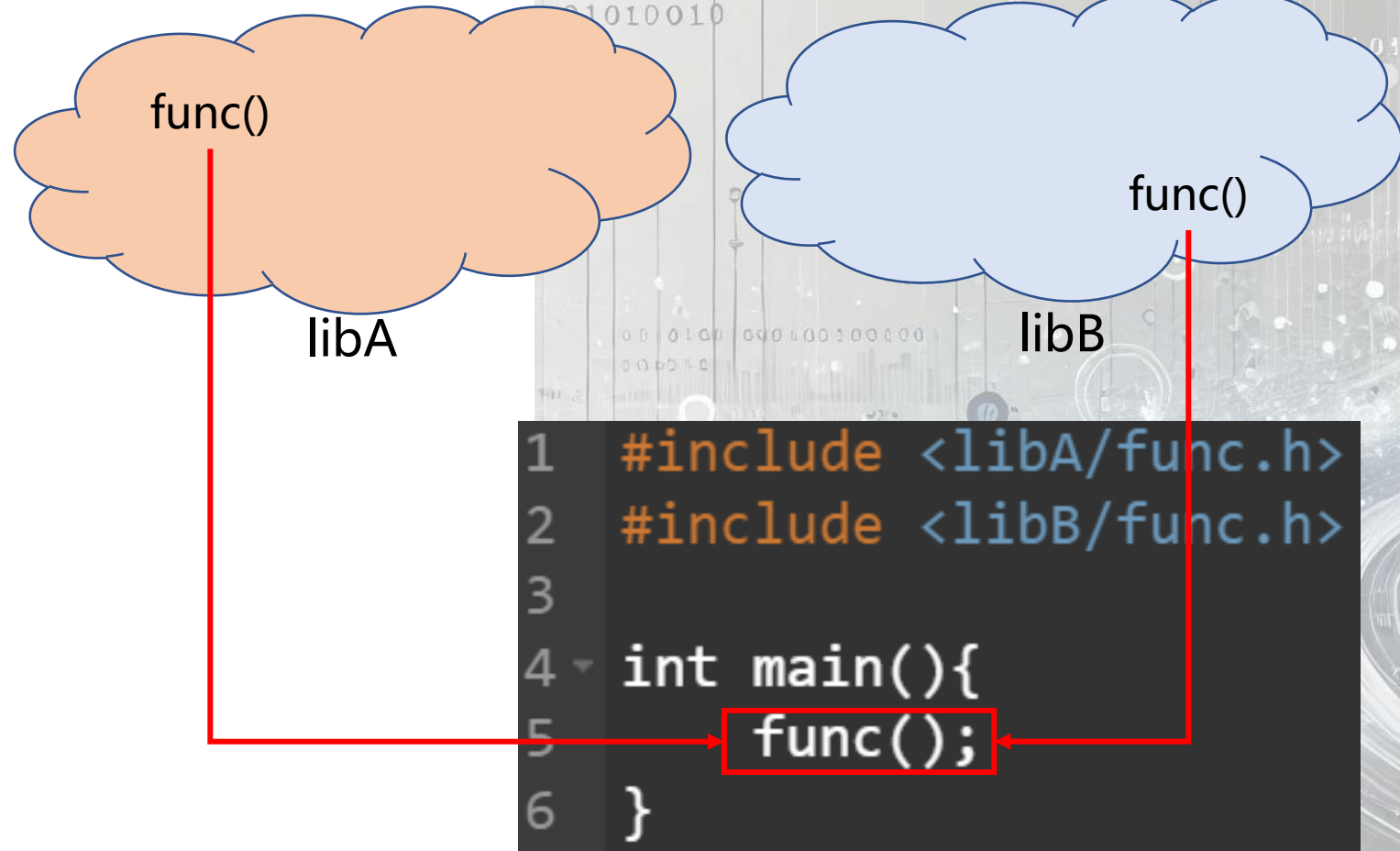
Scope 的概念出現

```
enum class PlayerStatus {  
    ALIVE = 0,  
    DEAD,  
}  
  
enum class EnemyStatus {  
    ALIVE = 0,  
    DEAD,  
}  
  
void func(PlayerStatus status) {  
    switch (status) {  
        case PlayerStatus::ALIVE: ... break;  
        case PlayerStatus::DEAD: ... break;  
    }  
}  
  
int main() {  
    func (PlayerStatus::ALIVE);  
    return 0;  
}
```



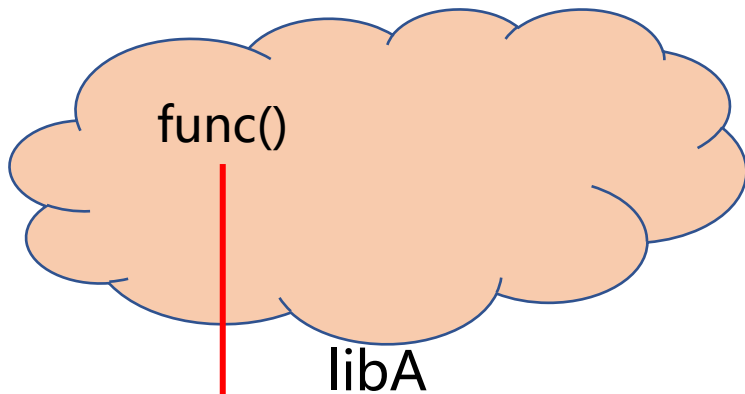
C++ 語法 - 基本 Enum Class篇

- 如果引入的兩個外部庫都使用到相同名稱的 function
 - 如 func()
- 編譯器要怎麼知道要拿誰的使用?
- 怎麼解決?
 - libA_func();
 - libB_func();

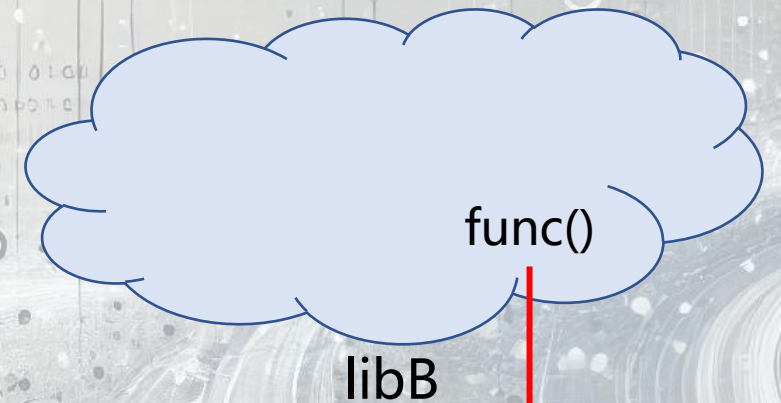


C++ 語法 - 基本 Namespaces篇

- 如果引入的兩個外部庫都使用到相同名稱的 function
 - 在C++你可以使用 Namespace
- Namespaces 用於解決命名衝突 (Naming conflicts) 的問題



```
1  #include <libA/func.h>
2  #include <libB/func.h>
3
4  int main(){
5      libA::func();
6      libB::func();
7  }
```



C++ 語法 - 基本 Namespaces篇

- 如果，整份程式碼中，都想指定使用 libA
- 每一個func()前面都要加上 libA:: 太痛苦了
 - 我們可以跟編譯器指定命名空間

```
1 #include <libA/func.h>
2 #include <libB/func.h>
3
4 namespace libA{
5     int main(){
6         func();
7         //LibB::func();
8     }
9 }
```

```
1 #include <libA/func.h>
2 #include <libB/func.h>
3
4 using namespace libA;
5
6 int main(){
7     func();
8     //LibB::func();
9 }
```

C++ 語法 - 基本 Function篇

- Statement 是一個 C++ 程式的表達方式
- 像這是一些常見的 statement, 在 cppreference 總共有定義 9 種

以下示意圖, 說明什麼是 statement

```
int main()  
{  
    int n = 1;           // declaration statement  
    n = n + 1;           // expression statement  
    std::cout << "n = " << n << '\n'; // expression statement  
    return 0;           // return statement  
}
```

C++ 語法 - 基本 Function篇

- Function 是一系列的 statements 所組成，可以抽象的理解成將一系列的指令包裝成一個功能
- 例如說：今天需要創建一個 Function 是“如何喝水”，所以它的操作步驟，也就是 statements 如下：
 1. 將手放到杯子上
 2. 杯子裡面裝滿水
 3. 杯子對接嘴巴
 4. 喝水
- 這樣一個一個步驟就稱為一個一個的 statement，再把他們包裝再一起就能稱作一個 Function



C++ 語法 - 基本 Function篇

```
void print_elem(char c) {  
    std::cout << c << std::endl;  
}  
  
int main() {  
  
    for (auto elem : {'a', 'p', 'p', 'l', 'e'}) {  
        print_elem(elem);  
    }  
  
    return 0;  
}
```

```
chenshiang@chenshiangdeMacBook-Air:~/Desktop/test/function  
➔ function git:(main) x clang++ -o main 01.cpp && ./main  
a  
p  
p  
l  
e  
➔ function git:(main) x
```

- 在 C++ 實作一個“印出陣列裡面資料”的 Function 示意
- 這個 print_elem 的 Function 把輸出進來的參數一個一個印出來 (c 為輸入變數, char 為變數的型別)
- 呼叫 Function 的地方, 能看到參數 elem 被一個個被丟進去 print_elem, 所以也就是說這個 Function 被呼叫了 5 次

C++ 語法 - 基本 Function篇

```
std::string is_even(int i) {  
    return i % 2 == 0 ? "yes" : "no";  
}  
  
int main() {  
    for (auto elem : {1, 2, 3, 4, 5, 6, 7}) {  
        std::cout << elem << ": " << is_even(elem) << std::endl;  
    }  
    return 0;  
}
```

```
chenshiang@chenshiangdeMacBook-Air:~/Desktop/test/function  
→ function git:(main) x clang++ -o main 02.cpp && ./main  
1: no  
2: yes  
3: no  
4: yes  
5: no  
6: yes  
7: no  
→ function git:(main) x
```

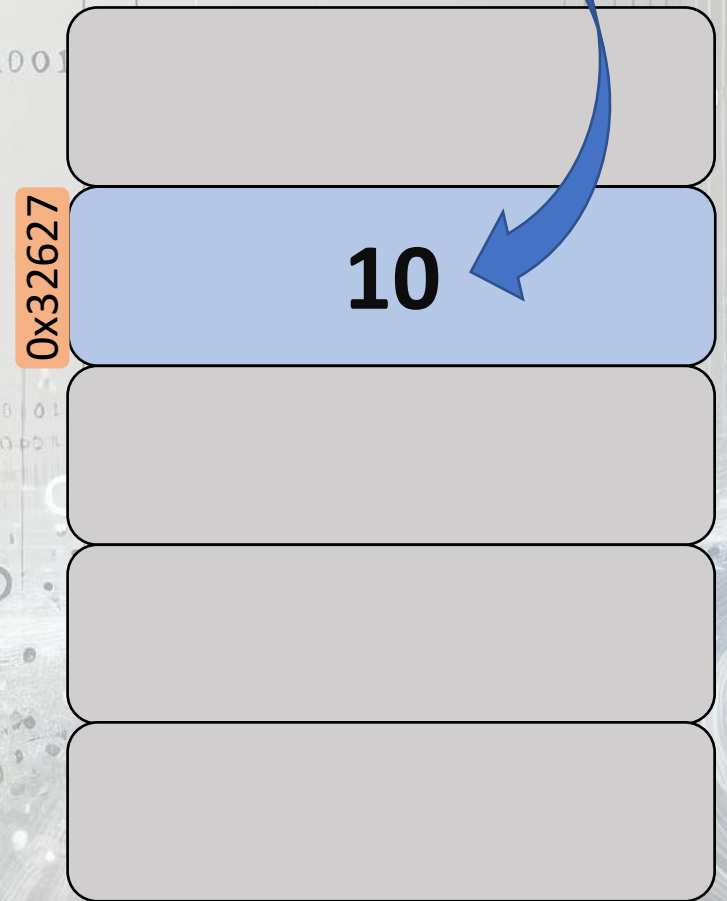
- 當然 Function 也能回傳物件資料, 只不過要在前方說明回傳的型別
- 在這個例子中, is_even 的 Function 判斷是否變數 i 為偶數, 使用了一個取餘數的運算子, 是的話就回傳'yes'不是就回傳'no'
- 在此我們結合前面所說的三元運算子

C++ 語法 - 基本 指標篇

- 語法

- 當我們宣告了一個變數並賦予值給它，實際上我們是在記憶體中找個位置，把這個值放進去
- 這個變數在記憶體哪個位置，是由編譯器與作業系統來配置的
- 指標是一種特殊的變數，它存儲的不是「變數的值」，而是「一個記憶體位置的地址」
- `int a=10;`
- `int* p = &a;` // p 是指向 int 的指標，存放 a 的地址
- `&a` 表示「取變數 a 的地址」
- p 存放這個地址

```
1 int a = 10;
```



Memory

C++ 語法 - 基本 指標篇

- & 運算子：取出變數被編譯器、作業系統所配置的記憶體位址 (&, Ampersand)
- * 運算子：取出指標變數所指向記憶體位址，裡面存的值，又稱解參照 (Dereference)

```
main.cpp
1  #include <iostream>
2
3  int main(){
4      int a = 10;
5      std::cout << &a;
6      return 0;
7  }
```

0x7ffea6fc2da4

```
main.cpp
1  #include <iostream>
2
3  int main(){
4      int a = 10;
5      std::cout << &a << std::endl;
6
7      int *ptr = &a;
8      *ptr = 5;
9      std::cout << *ptr << std::endl;
10     return 0;
11 }
```

0x7ffe674efd7c
5

```
1 int a = 10;
```

0x32627

10

Memory

C++ 語法 - 基本 Reference篇

- 引用/參考，是變數的一個「別名」，它本身不是(新)變數，不佔新記憶體，而是直接綁定到某個已存在的變數
- 使用上：
 - 必須初始化（引用必須引用已經存在的變數）
 - 不能改變綁定對象（一旦 reference 指向某個變數，就不能再「轉向」另一個變數）
 - 沒有額外的記憶體成本（不複製，直接操作原變數）

```
1  #include <iostream>
2
3  int main()
4  {
5      int a = 10;
6      int& r = a;    // r 是 a 的參考（別名）
7
8      r = 20;        // 等同於 a = 20
9      std::cout << a;    // 輸出 20
10 }
```

Type的一部分

Example 比較

```
1 #include <iostream>
2 using namespace std;
3
4 void addTenWithPointer(int* p) {
5     if (p != nullptr) {
6         *p += 10;
7     }
8 }
9
10 int main() {
11     int x = 5;
12     addTenWithPointer(&x);
13     cout << x << endl;
14 }
```

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 void printString(string s) {
6     cout << s << endl;
7 }
8
9 int main() {
10     string bigData = "這是一個非常非常非常長的字串.....";
11     printString(bigData);
12 }
```

```
1 #include <iostream>
2 using namespace std;
3
4 void addTenWithReference(int& r) {
5     r += 10;
6 }
7
8 int main() {
9     int x = 5;
10    addTenWithReference(x);
11    cout << x << endl;
12 }
```

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 void printString(const string& s) {
6     cout << s << endl;
7 }
8
9 int main() {
10     string bigData = "這是一個非常非常非常長的字串.....";
11     printString(bigData);
12 }
```

C++ 語法 - 基本 指標篇

- 指標章節具有一定的難度
 - 什麼能當變數，就能操作Pointer
 - Function Pointer
 - Array...
 - Value...

C isn't that hard:

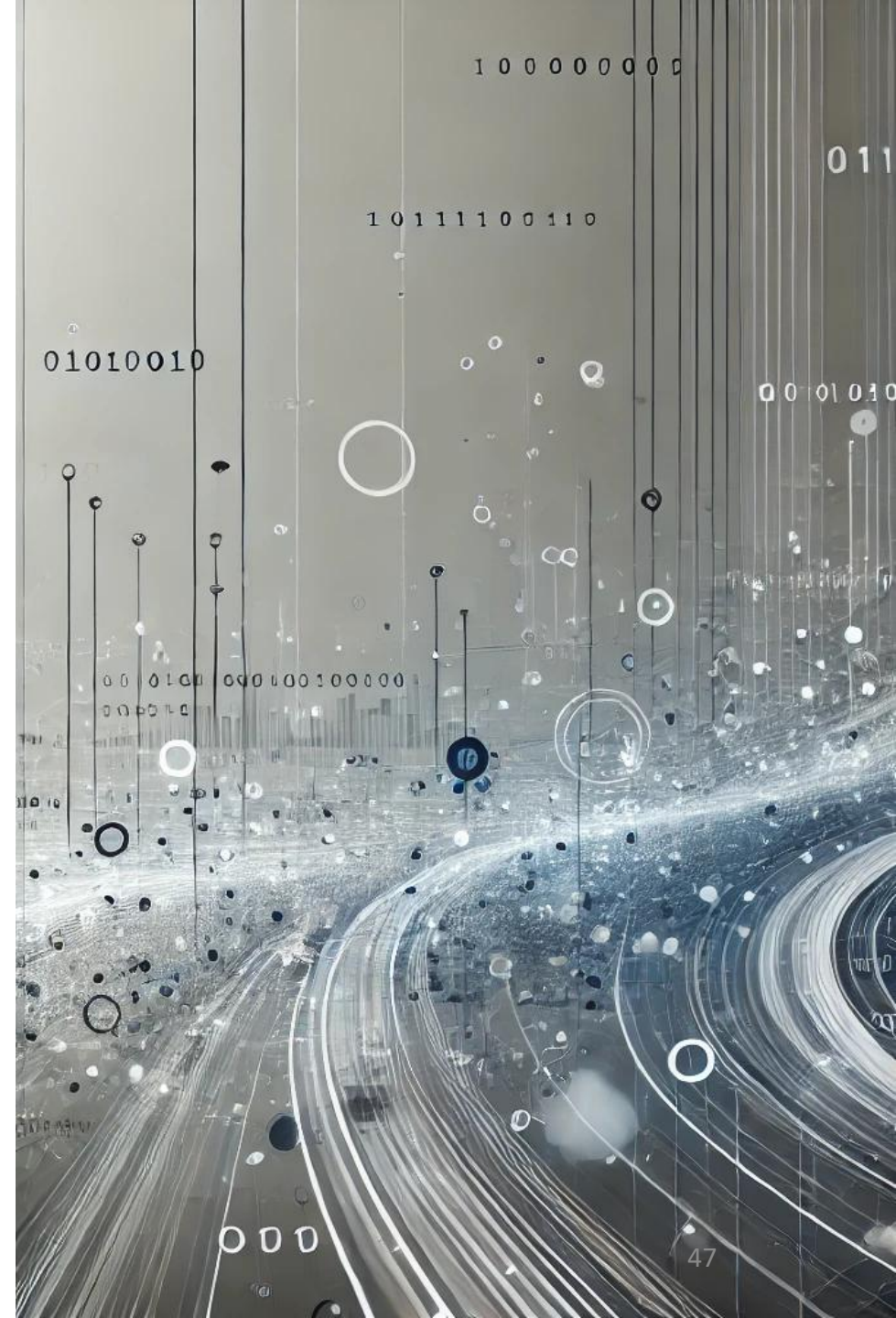
```
void (*(f[])) ();
```

Defines *f* as an array of unspecified size of pointers to functions that return pointers to functions with a return type of *void*.



主題四：

Compiler 的介紹



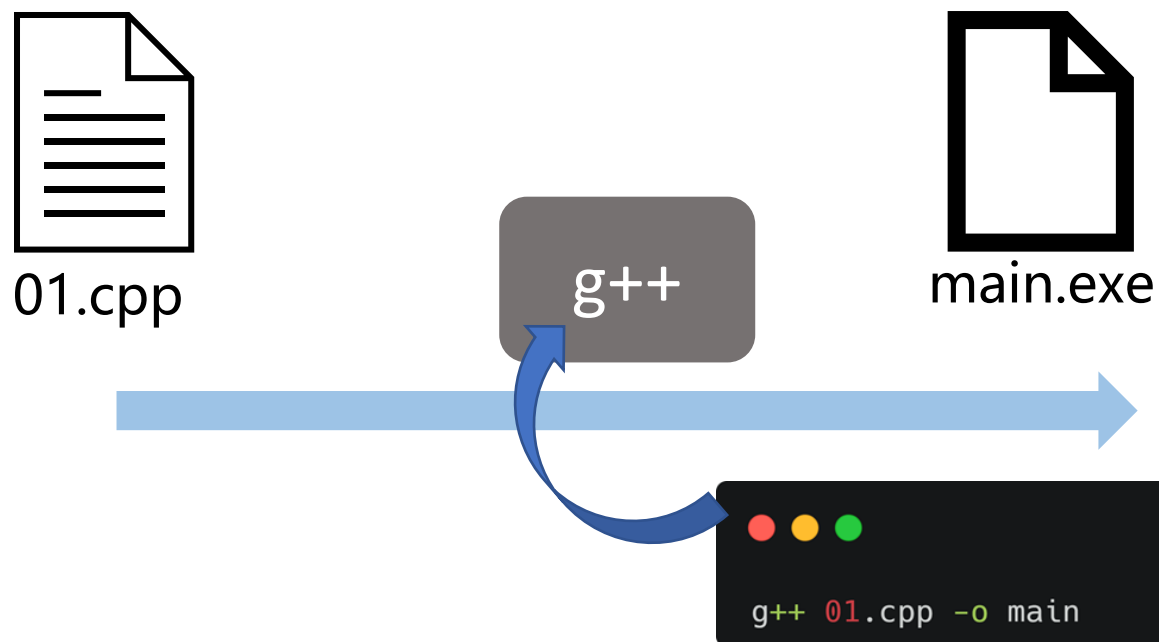
什麼是 Compiler

- Compiler 是負責把一個高階語言的 source code (例如C、C++、Java 等) 轉換成低階 target code 的工具。這個 target code 可能是：
 - 機器碼 (Machine code) : 能直接在 CPU 上執行
 - 組合語言 (Assembly code) : 通常是比較接近機器碼的中間表示, 再透過 assembler 轉換成機器碼
 - 中間表示 (Intermediate Representation, IR) : 像 LLVM IR, 之後再由後端或虛擬機轉換成特定平台的機器碼
- 主流的 Compiler :
 - Clang – LLVM
 - GCC - GNU Compiler Collection



什麼是 Compiler

- 接下來我們嘗試使用 g++ 編譯 source code 成機器碼，屬於可執行檔 (Executable binary)。它的示意圖像是這個樣子



一個簡單的命令

```
#include <iostream>

int main() {

    std::cout << "Hello, world!\n";

    return 0;
}
```

- `g++ 01.cpp -o main`

```
chenshiang@chenshiangdeMacBook-Air:~/Desktop/test/compiler2
→ compiler2 git:(main) x g++ 01.cpp -o main
→ compiler2 git:(main) x ./main
Hello, world!
→ compiler2 git:(main) x
```

- 這是一個簡單的 Hello world 程式,

- 現在用命令的方式去執行他

- 可以把指令分成幾個部分

- `g++`: 使用的編譯器

- `01.cpp`: 所需要編譯的檔案

- `-o main`: 編譯後執行檔案的名稱

- 接下來編譯完成後, 我們就可以執行了, 在終端機輸入 `./main`

Header file

- 假設有三個檔案分別為：
 - hello.cpp
 - hello.hpp
 - main.cpp
- 我們會需要三個檔案進行編譯, 了解並思考為什麼需要 Header file, 在其他語言 Python, Java, Javascript 等都沒有看到類似的東西, 而偏偏只有C/C++獨有



Header file

- 假設今天去了一家高檔餐廳, 想要點幾份東西來吃, 因此你會根據菜單來選擇吃的東西, 而廚師會根據菜單做出相對應得食物
- 因此得到一個結論是**菜單只是一個預覽**, 並不存在任何的實作方式 (菜單不存在像食譜般教導你如何做出這項食物)



你



菜單



廚師

Header file

- 程式碼範例

```
1 //main.cpp
2 #include <iostream>
3 #include "hello.hpp"
4
5 int main(void)
6 {
7
8     hello();
9
10    return 0;
11 }
```



你

```
1 //hello.hpp
2 #ifndef HELLO_HPP
3 #define HELLO_HPP
4
5 void hello();
6
7 #endif
```



菜單

```
1 //hello.cpp
2 #include "hello.hpp"
3 #include <iostream>
4
5 void hello(){
6
7     std::cout << "Hello world!";
8
9 }
```



廚師

- 抽象概念

Header file

- hello.hpp 通常被稱為 Header file, 中文為「標頭檔」
- 通常副檔名為
 - .h 在 C 語言 以及 .hpp 在 C++ 語言
 - 一般來說, main.cpp、hello.hpp、以及 hello.cpp 這三個檔案**不一定**會放相同資料夾中

檔案格式:

```
./
├── hello.cpp
├── hello.hpp
└── main.cpp
```

Header file

- 如何用命令編譯
- 可以融會貫通地用剛剛學的指令進行多重檔案的編譯

```
gcc -o main main.c hello.c
```

- 更推薦將他分為三次編譯

```
gcc -o main.o -c main.c  
gcc -o hello.o -c hello.c  
gcc -o main main.o hello.o
```



Header file

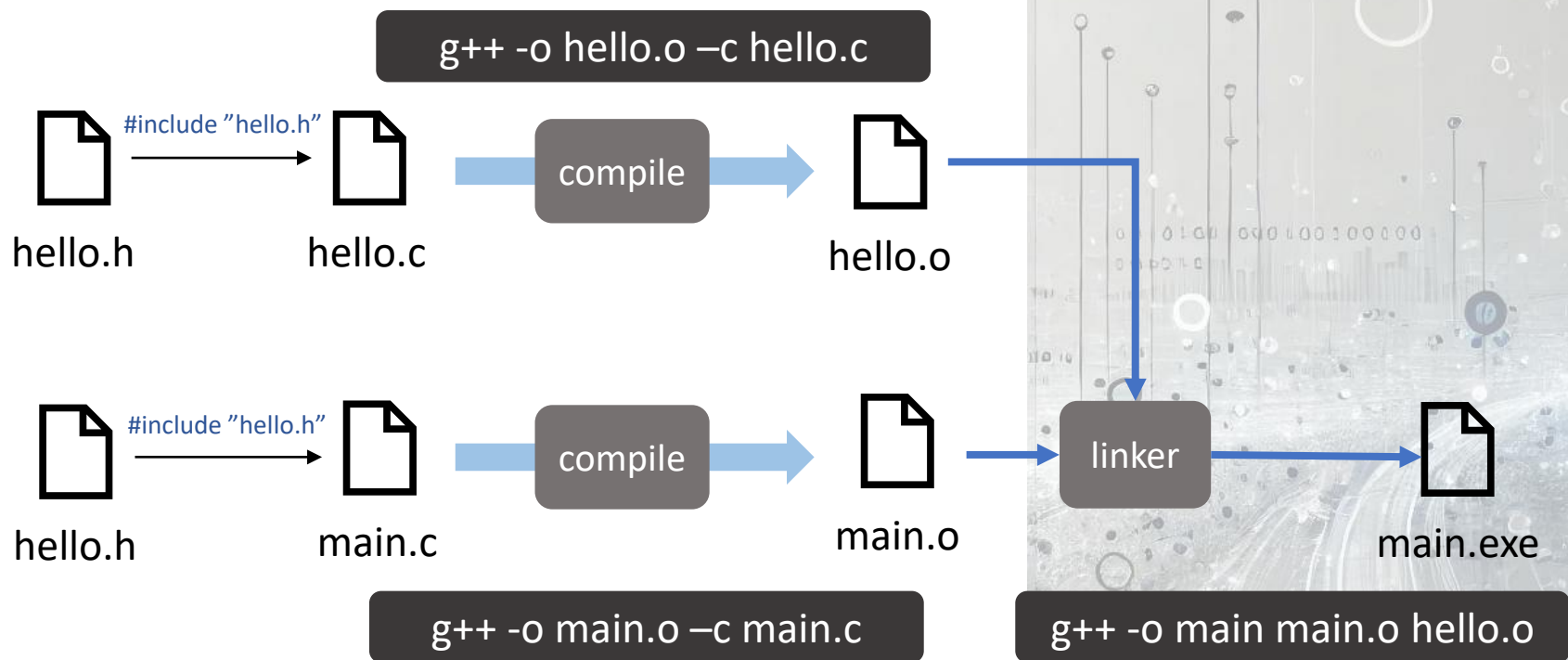
- 當面對超大型專案的時候，往往會有很多的檔案需要編譯，因此可以分為兩段式編譯：
 - 先把 source code 編譯成一種 object code（又稱為目的碼）
 - 等到其他剩下的檔案都編譯完成後，再把所有的目的碼連結在一起，得到執行檔
- 在面對更新code以後，進行重新編譯時，不用全部重新做
 - 只需要重新編譯那些與更新檔案有相依性的就好
- 常見的 *.o 是稱為 object file，也就是上述目的碼的概念



```
gcc -o main.o -c main.c      # *.c to *.o
gcc -o hello.o -c hello.c    # *.c to *.o
gcc -o main main.o hello.o   # *.o to main.exe
```

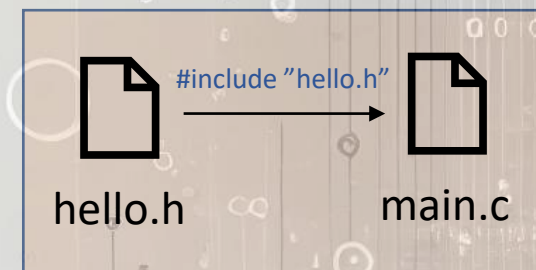

Header file

- 以下是指令的示意圖，可以大概的看到一個編譯器的流程是如何運作的



指導語句 Directives

- 當我們在 main.c 或者其他 .c 或 .cpp 檔案使用 # 的關鍵字時, 會在編譯前對 source code 做一些預處理
- #include "<檔案名稱>" 的運作方式
 - 插入特定的標頭檔到程式碼中
- 常見的 Directives:
 - #include <iostream>
 - #include <[file]>
 - #define [key] [value]
 - Include guard



hello.h

```
// hello.h

#ifndef HELLO_H
#define HELLO_H

void hello();

#endif
```

main.c

```
// main.c

#include "hello.h"

int main(void) {

    hello();

    return 0;
}
```

Header file

- 當預處理器執行到 #include 的時候, 會在原有的地方「展開」header file 裡面的內容
- 示意圖如右所示

hello.h

```
// hello.h

#ifndef HELLO_H
#define HELLO_H

void hello();

#endif
```

main.c

```
// main.c

#include "hello.h"

int main(void) {

    hello();

    return 0;

}
```

Result

```
void hello();

int main(void) {

    hello();

    return 0;

}
```

Header file

- 為什麼在 header file 的地方會出現這麼多的陌生的指導語句，如果把它們移除會發生什麼事情？

main.c

```
// main.c

#include "hello.h"

int main(void) {

    hello();

    return 0;
}
```

hello.h

```
// hello.h

#ifndef HELLO_H
#define HELLO_H

void hello();

#endif
```

hello.c

```
// hello.c

#include "hello.h"

#include <stdio.h>

void hello() {
    printf("hello world\n");
}
```


Header file

- 如果在兩份 Header file 有著串連的關係
- 例如：在 main.c 裡面導入 02_00.h 和 02_01.h, 其中 02_01.h 也會導入 02_00.h 的情況
- **會發生什麼問題嗎？**

```
02_01.h
#include "02_00.h"
int add(int a, int b);
```

```
02_00.h
int minus(int a, int b);
```

```
main.c
#include "02_00.h"
#include "02_01.h"

int main() {
    int a = 1, b = 1;
    int sum = add(a, b);
    int diff = minus(a, b);

    return 0;
}
```

Header file

- 完成導入 02_00.h 之後
- 可以觀察 02_01.h 導入的時候，發生**同一個函數被重複引用**

```
02_00.h

int minus(int a, int b);
```

```
02_01.h

#include "02_00.h"

int add(int a, int b);
```

Step 1: 導入 02_00.h

Step 2: 導入 02_01.h

```
main.c

#include "02_00.h"
#include "02_01.h"

int main() {
    int a = 1, b = 1;
    int sum = add(a, b);
    int diff = minus(a, b);

    return 0;
}
```

```
main.c

// main.c
int minus(int a, int b); // #include "02_00.h"
#include "02_01.h"

int main() {
    int a = 1, b = 1;
    int sum = add(a, b);
    int diff = minus(a, b);

    return 0;
}
```

Step 2: main.c

```
main.c

// main.c
int minus(int a, int b); // #include "02_00.h"
int minus(int a, int b); // #include "02_01.h"
int add(int a, int b); // #include "02_01.h"

int main() {
    int a = 1, b = 1;
    int sum = add(a, b);
    int diff = minus(a, b);

    return 0;
}
```

引用防護 (Include Guard)

```
02_01.c
#include "02_01.h"

int add(int a, int b);

02_01.h
#ifndef HEADER_FILE_01
#define HEADER_FILE_01

#include "02_00.h"

int add(int a, int b);

#endif // HEADER_FILE_01
```

```
02_00.c
#include "02_00.h"

int minus(int a, int b);

02_00.h
#ifndef HEADER_FILE_01
#define HEADER_FILE_01

int minus(int a, int b);

#endif // HEADER_FILE_01
```

```
main.c
#include "02_00.h"
#include "02_01.h"

int main() {
    int a = 1, b = 1;
    int sum = add(a, b);
    int diff = minus(a, b);

    return 0;
}
```

• 因此為了同一個函數被重複引用, 因此我們需要設定一個檔案只能編譯一次

- 引用防護的流程是: 如果這個參數沒有被定義過 (`#ifndef <參數>`), 那就先定義這個參數 (`#define <參數>`), 再繼續編譯, 後而結束整個判斷 (`#endif`)
- 反之, 如果這個參數有被定義, 就不再重複編譯

指引語法 (#pragma) 的用法

```
#include "02_00.h"
#pragma once
#include "02_00.h"

int add(int a, int b);
```

```
#include "02_01.h"
#pragma once
int minus(int a, int b);
```

```
#include "02_00.h"
#include "02_01.h"

int main() {
    int a = 1, b = 1;
    int sum = add(a, b);
    int diff = minus(a, b);

    return 0;
}
```

- pragma 是用來告知編譯器某些特殊指示，經常出現在她/他人的 Project 內
- 例如 #pragma once 是要告訴編譯器：「只能編譯這個檔案一次」
- 但因為這個用法並沒被規範在 C++ 語言標準規範裡面，#pragma once 這個語法並不是所有編譯器都有實作這個功能
- 也就是說如果使用比較小眾的編譯器，有可能會發現這個語法並不支援

建構工具的應用

- 可以思考一個問題：在超大型專案內這樣命令是不是也顯得複雜, 當原始碼的檔案非常多的時候, 就需要自己觀察哪些檔案是有被更改的, 然後在使用相對應的命令

```
gcc -o main.o -c main.c      # *.c to *.o
gcc -o hello.o -c hello.c    # *.c to *.o
gcc -o main main.o hello.o  # *.o to main.exe
```

- 因此希望能有個工具能自己幫我們自動追蹤所撰寫的專案, 而且只需要下一次命令就能完成編譯成一個執行檔案。

```
// main.c

#include "hello.h"

int main(void) {

    hello();

    return 0;
}
```

```
// hello.h

#ifndef HELLO_H
#define HELLO_H

void hello();

#endif
```

```
// hello.c

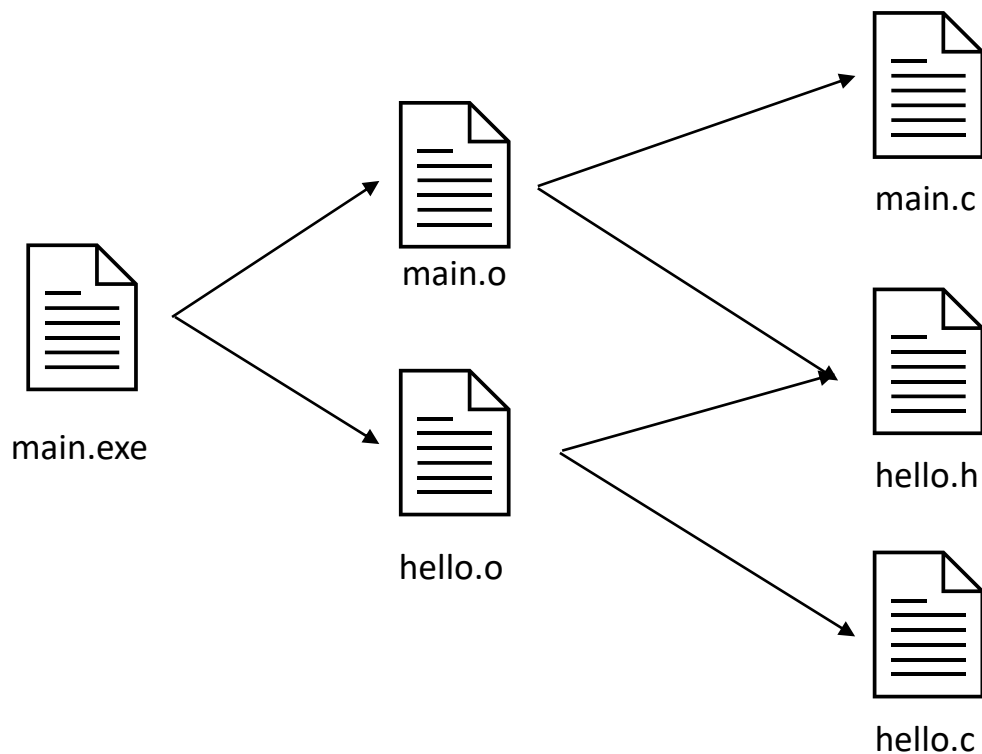
#include "hello.h"

#include <stdio.h>

void hello() {
    printf("hello world\n");
}
```

建構工具的應用

- 對於我們使用的建構工具，它所看到的狀態就是像這樣，也就是內部會有一棵樹用來儲存專案中每一個檔案的關係圖



```
// main.c

#include "hello.h"

int main(void) {

    hello();

    return 0;
}
```

```
// hello.h

#ifndef HELLO_H
#define HELLO_H

void hello();

#endif
```

```
// hello.c

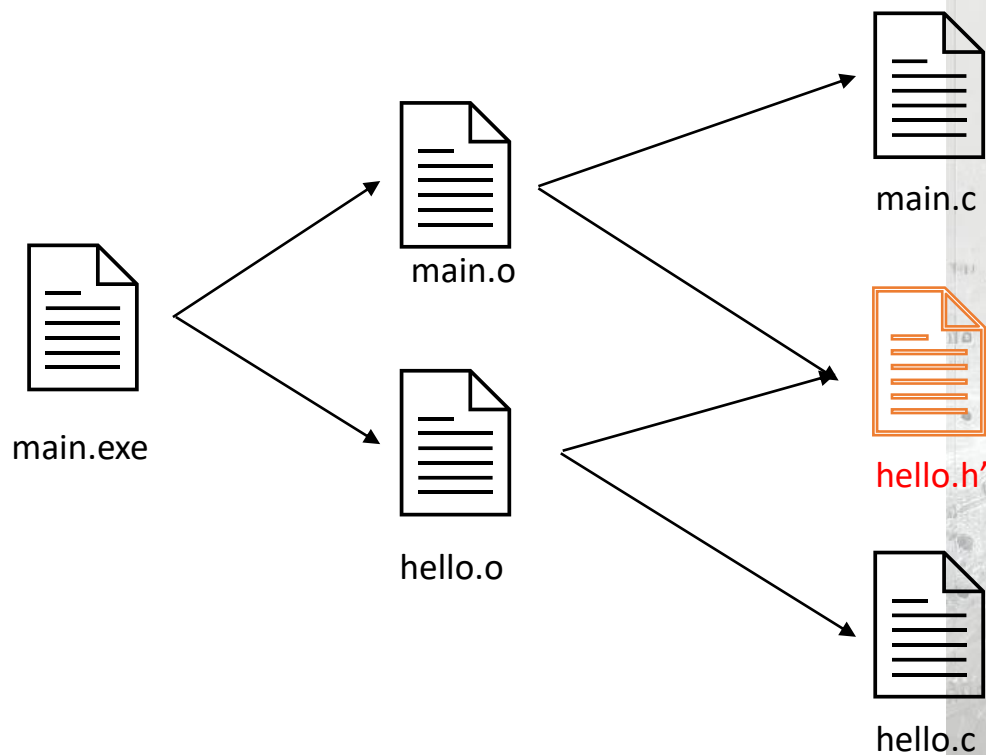
#include "hello.h"

#include <stdio.h>

void hello() {
    printf("hello world\n");
}
```

建構工具的應用

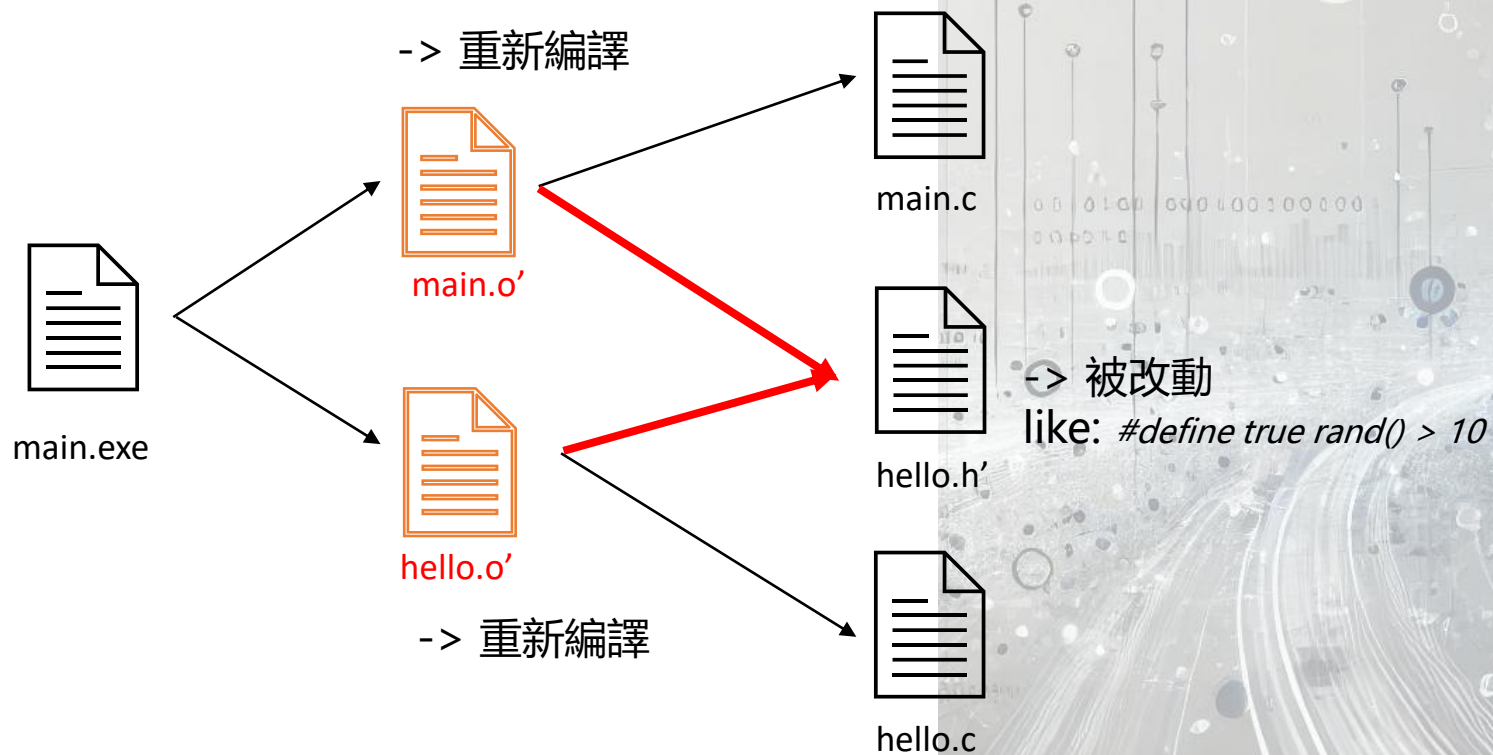
- 假設有一個檔案被改動，那麼建構工具將會記錄下那個檔案，並重新編譯該檔案



-> 被改動
like: `#define true rand() > 10`

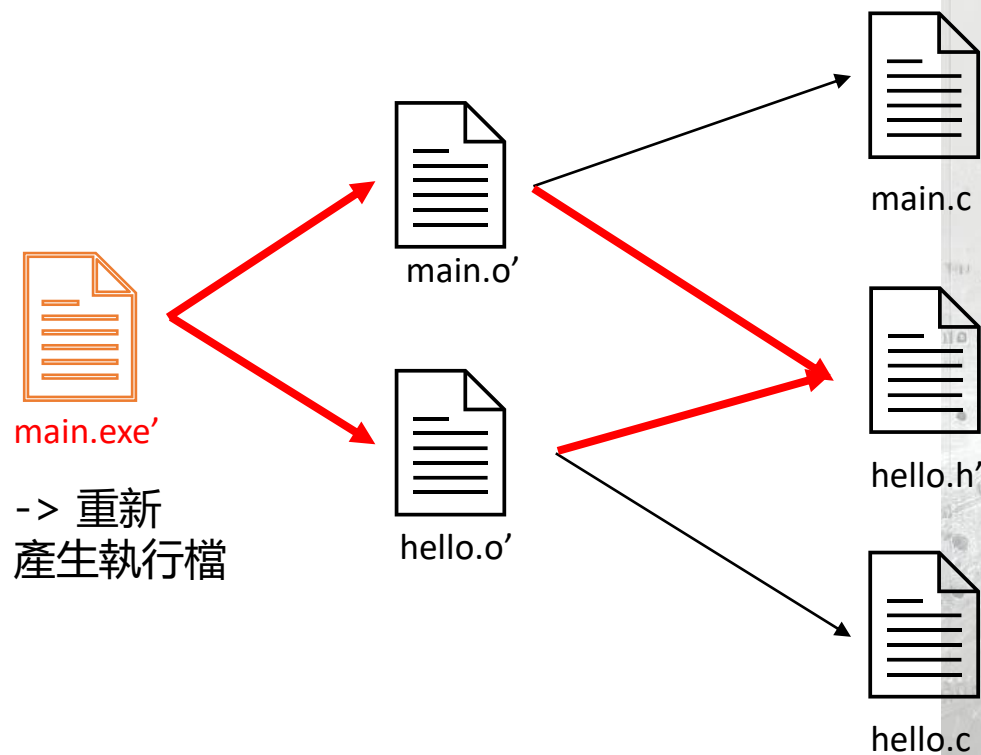
建構工具的應用

- 與變動過的檔案有相依性的檔案，都得重新編譯一次



建構工具的應用

- 然後連結，重新產生執行檔



-> 被改動
like: `#define true rand() > 10`

建構工具的應用

makefile

```
# Makefile

CC := gcc

INC := \
    hello.h \

OBJ := \
    main.o \
    hello.o \

.PHONY: all clean
all: main

clean:
    rm -rf *.o main

main: $(OBJ)
    $(CC) -o $@ $^

%.o: %.c $(INC)
    $(CC) -o $@ $< -c
```

- 在此介紹一個工具稱為 Make, 他能解決我們上述說明的問題, 自動追蹤哪些檔案是被修改的, 並且在編譯的時候自動幫我們完成所撰寫的命令
- 本課程中, 不會說明建構工具的語法
 - 已有許多參考資料
 - 左圖為一個 makefile 的寫法, 用於編譯之前所說的三個檔案

建構工具的應用

makefile

```
# Makefile

CC := gcc

INC := \
    hello.h \

OBJ := \
    main.o \
    hello.o \

.PHONY: all clean
all: main

clean:
    rm -rf *.o main

main: $(OBJ)
    $(CC) -o $@ $^

%.o: %.c $(INC)
    $(CC) -o $@ $< -c
```

- 必須安裝 make 的套件後，使用這個命令能完成整個建構

```
make all
```

- 但就像微軟所提供的系統，有時候你會不知道他哪裡有問題，然後重開機解決一切問題一樣。

- 你可以運用這個指令清理掉 Make 所建構的檔案

```
make clean
```

建構工具的應用

- 另外一套建構工具 CMake
 - Make 工具都只能在 Unix 環境中編譯,
 - CMake 為一個跨平台的建構工具
- 建構工具 CMake, 也就是你在作業所看到本課程所使用的建構方式



CMake



建構工具的應用

CMakeLists.txt

```

# CMakeLists.txt

cmake_minimum_required(VERSION 3.0.0)
project(hello)

add_executable(main
    hello.c
    main.c

    hello.h
)
```

- 你可以透過CMakeLists.txt 設定整個專案



```
cmake -B build
```

- 接下來是開始編譯



```
cmake --build build
```

- 當然也可以清空整個檔案再重新編譯



```
cmake --build build --target clean
```